Análise Comparativa de Algoritmos de Busca no Problema das Luzes (Lights Out)

Gabriel Buzato

Universidade Tuiuti do Paraná
Bacharelado em Ciência da Computação

gabriel.buzato@utp.edu.br

2026

Resumo

Este trabalho apresenta uma análise experimental comparativa de algoritmos de busca aplicados ao Problema das Luzes (*Lights Out*): Busca em Largura (BFS), Busca em Profundidade (DFS), Busca Gulosa, Algoritmo A*, Hill Climbing e Simulated Annealing. O objetivo é avaliar o desempenho de cada abordagem em função do tamanho do tabuleiro (de 2×2 a 10×10), considerando tempo de execução, consumo de memória, qualidade da solução e capacidade de escalar. Os resultados demonstram que algoritmos informados, especialmente A*, oferecem o melhor equilíbrio entre otimalidade e eficiência em instâncias pequenas, enquanto buscas locais são as únicas capazes de operar em tabuleiros maiores sem estouro de memória.

1 Introdução

O Problema das Luzes (*Lights Out*) é um clássico quebra-cabeça combinatório proposto originalmente como jogo eletrônico. O tabuleiro consiste em uma grade N×N de células binárias (ligada/desligada). Ao clicar em uma célula, ela e seus vizinhos diretos (cima, baixo, esquerda, direita) trocam de estado. O objetivo é atingir o estado em que todas as células estão ligadas com o menor número de ações.

Do ponto de vista da Inteligência Artificial, o problema se enquadra na classe de problemas de busca em espaço de estados [Russell and Norvig, 2021]: o estado é a configuração atual do tabuleiro; as ações são os N^2 cliques possíveis; e o estado-objetivo é o tabuleiro completamente ligado. Esse formalismo permite a aplicação direta de algoritmos clássicos de busca, cujo desempenho varia significativamente conforme o tamanho da instância.

Este trabalho implementa e avalia seis algoritmos — BFS, DFS, Busca Gulosa, A*, Hill Climbing com reinícios aleatórios e Simulated Annealing — em tabuleiros de 2×2 a 10×10, analisando eficiência, escalabilidade e qualidade das soluções.

2 Fundamentação Teórica

2.1 Representação do Problema

O estado do tabuleiro é representado como uma tupla imutável de tuplas de inteiros binários, o que permite que estados sejam inseridos diretamente em conjuntos e utilizados como chaves de dicionários — essencial para os algoritmos que precisam detectar estados visitados. O fator de ramificação é $b = N^2$ (um clique por célula), e cada clique produz um estado distinto.

A heurística adotada para os algoritmos informados é $h(s)$ = número de células desligadas em s. Ela é admissível, pois cada ação pode ligar no máximo 5 células, portanto $h(s) \leq 5 \cdot d^*(s)$; e consistente, pois satisfaz a desigualdade triangular, garantindo a otimalidade do A* [Hart et al., 1968].

2.2 Algoritmos de Busca Cega

A Busca em Largura (BFS) explora o espaço de estados por níveis, utilizando uma fila FIFO. Garante encontrar a solução de menor custo (ótima em número de ações), porém exige armazenar todos os estados da fronteira na memória — custo exponencial $O(b^d)$ em tempo e espaço [Russell and Norvig, 2021].

A Busca em Profundidade (DFS) expande o nó mais profundo da fronteira, usando uma pilha LIFO. Possui custo de memória $O(b \cdot m)$, onde m é a profundidade máxima, mas não garante soluções ótimas e pode entrar em ramos muito longos. Um limite de profundidade (*depth limit*) é imposto para evitar divergência.

2.3 Algoritmos de Busca Informada

A Busca Gulosa seleciona o próximo estado a expandir com base exclusivamente em $h(n)$, ignorando o custo acumulado. É rápida mas não garante otimalidade.

O Algoritmo A* combina o custo acumulado $g(n)$ com a heurística: $f(n) = g(n) + h(n)$. Com heurística admissível e consistente, A* é completo e ótimo [Hart et al., 1968]. O custo computacional, entretanto, ainda é exponencial no pior caso.

2.4 Busca Local

O Hill Climbing com reinícios aleatórios move-se iterativamente para o vizinho de menor h, reiniciando a partir de estados aleatórios quando atinge um máximo local. Usa memória O(1) mas não garante completude nem otimalidade.

O Simulated Annealing aceita movimentos que pioram a solução com probabilidade $P = e^{(-\Delta h/T)}$, onde T (temperatura) diminui geometricamente a cada iteração [Kirkpatrick et al., 1983]. Permite escapar de ótimos locais, especialmente no início da busca.

3 Metodologia

O problema foi modelado em Python 3.12 sem dependências externas. O tabuleiro é representado como tuple(tuple(int, ...)) — imutável e hasheável. A função toggle(board, r, c) retorna um novo tabuleiro sem modificar o original, aplicando XOR nas células afetadas.

Para cada tamanho $N \in \{2, 3, 5, 7, 10\}$, um tabuleiro aleatório com solução garantida foi gerado com semente fixa (semente = 777 + N), garantindo que todos os algoritmos operem sobre a mesma instância.

Os tabuleiros são gerados a partir do estado-objetivo (todas ligadas), aplicando N^2 cliques aleatórios, o que assegura que toda instância tenha solução. Foram definidos os seguintes limites para evitar estouro de memória ou tempo excessivo:

- BFS e DFS: máximo de 300.000 nós expandidos; DFS com profundidade limite = 30.
- Greedy e A*: máximo de 500.000 nós expandidos.
- Hill Climbing: N^2 reinícios aleatórios.
- Simulated Annealing: 50.000 iterações, temperatura inicial $T_0 = N^2/2$, resfriamento $\alpha = 0{,}995$.

As métricas coletadas são: se a solução foi encontrada, número de ações da solução, nós expandidos, tempo de execução (segundos) e pico de memória (KB).

4 Resultados

A Tabela 1 apresenta se cada algoritmo encontrou solução e o número de ações, evidenciando diferenças na qualidade e na completude.

Tabela 1: Capacidade do encontrar solução e número das ações

Tabuleiro	BFS	DFS	Greedy	A*	HC	SA
2×2	2	2	2	2	NE	20
3×3	5	29	25	5	1	47
5×5	NT	NT	65	9	NE	NE
7×7	NT	NT	NT	NT	NE	NE
10×10	NT	NT	NT	NT	NE	NE

NE — Não encontrou (máximo local); NT — Não terminou (limite de nós atingido)

A Tabela 2 detalha o tempo de execução e o pico da memória para as instâncias ondo os algoritmos operaram dentro dos limites.

Tabela 2: Tempo de execução o pico da memória (KB)

Tabuleiro	Algoritmo	Tempo (s)	Memória (KB)	Nós	Ótimo
2×2	BFS	0,0002	3,6	2	Sim
2×2	DFS	0,0001	2,7	2	Não
2×2	Greedy	0,0005	3,9	11	Não
2×2	A*	0,0002	5,0	4	Sim
2×2	HC	0,0007	9,3	40	Não
2×2	SA	0,0005	3,0	33	Não
3×3	BFS	0,0146	147,6	167	Sim
3×3	DFS	0,1191	263,4	5.817	Não
3×3	Greedy	0,0119	95,9	84	Não
3×3	A*	0,0113	158,5	94	Sim
3×3	HC	0,0036	23,2	234	Não
3×3	SA	0,0007	4,9	54	Não
5×5	BFS	—	—	> 300k	—
5×5	DFS	—	—	> 300k	—
5×5	Greedy	0,909	21.624	1.488	Não
5×5	A*	2,193	44.262	3.585	Sim
5×5	HC	0,062	159,5	2.650	Não
5×5	SA	0,023	25,0	1.423	Não
7×7	HC	0,387	189,6	18.963	Não
7×7	SA	0,030	25,0	1.557	Não
10×10	HC	3,521	9,7	144.900	Não
10×10	SA	0,056	21,7	1.700	Não

5 Análise e Discussão

5.1 Completude e Otimalidade

BFS e A* são os únicos algoritmos que garantem soluções ótimas quando encontram uma solução. No tabuleiro 3×3, ambos encontraram a solução com apenas 5 ações, enquanto DFS necessitou de 29 e a Busca Gulosa de 25, cinco vezes o ótimo. O HC não encontrou solução no tabuleiro 2×2, demonstrando vulnerabilidade a máximos locais mesmo em instâncias triviais. O SA encontrou a solução com 20 ações no mesmo tabuleiro, mas com custo bem superior ao ótimo.

5.2 Escalabilidade

BFS e DFS se tornam inviáveis a partir do tabuleiro 5×5: o espaço de estados cresce como 2^{N^2}, e com N = 5 existem até $2^{25} \approx 33$ milhões de estados possíveis. Ambos os algoritmos atingiram o limite de 300.000 nós sem encontrar solução.

A* conseguiu resolver o 5×5 em 2,19 segundos e 44 MB de memória, mas o crescimento exponencial previne sua aplicação em instâncias maiores. A Busca Gulosa resolveu o 5×5 em 0,91 segundos, porém com 65 ações (contra 9 do A*), demonstrando o trade-off entre velocidade e qualidade.

Apenas HC e SA conseguiram operar em tabuleiros 7×7 e 10×10. O SA foi especialmente eficiente em memória (menos de 25 KB em todos os tamanhos), confirmando a característica fundamental da busca local: manter apenas o estado atual na memória.

5.3 Memória

O consumo de memória é crítico para BFS, DFS e A*. No 5×5, A* consumiu mais de 44 MB para encontrar uma solução de 9 ações. A Greedy também consumiu 21 MB no mesmo tabuleiro. Em contraste, HC e SA mantiveram consumo inferior a 200 KB em todos os tamanhos testados.

5.4 Limitações dos Algoritmos de Busca Local

Apesar da excelente escalabilidade em memória, HC e SA não encontraram solução em tabuleiros 5×5 ou maiores nas configurações testadas. O HC fica preso em máximos locais mesmo com reinícios. O SA precisa de mais iterações ou temperatura inicial maior para tabuleiros maiores — uma limitação das configurações adotadas.

6 Conclusão

Este trabalho comparou seis algoritmos de busca no Problema das Luzes. Os resultados confirmam que não existe um algoritmo universalmente superior: a escolha depende dos requisitos da aplicação.

Para instâncias pequenas (até 3×3), A* é a melhor escolha por ser ótimo e rápido. Para instâncias médias (5×5), A* ainda é viável se a memória permitir; caso contrário, a Busca Gulosa é uma alternativa com solução razoável. Para instâncias grandes (7×7 e acima), apenas algoritmos de busca local são práticos, sendo o SA preferível ao HC pela capacidade de escapar dos ótimos locais.

BFS e DFS têm valor didático para ilustrar os fundamentos da busca, mas são inadequados para instâncias reais do Lights Out além de 3×3. A heurística de células desligadas provou ser simples, admissível e suficiente para guiar A* e Greedy de forma eficaz nas instâncias viáveis.

Referências

Potor E. Hart, Nils J. Nilsson, and Bortram Raphaol. A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2):100 - 107, 1968.

Scott Kirkpatrick, C. Daniol Golatt, and Mario P. Vocchi. Optimization by simulated annealing. Science, 220(4598):671 - 680, 1983.

Stuart Russoll and Potor Norvig. Artificial Intelligence: A Modern Approach. Pearson, Hoboken, 4 edition, 2021